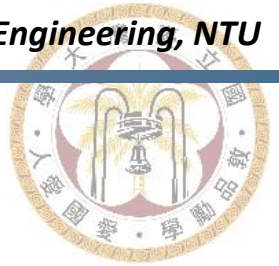
# Advanced Computer-Aided VLSI System Design

# Homework 3: Low Power Matrix Accelerator

*Graduate Institute of Electronics Engineering, National Taiwan University*

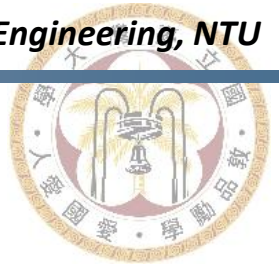


NTU GIEE



# Goals

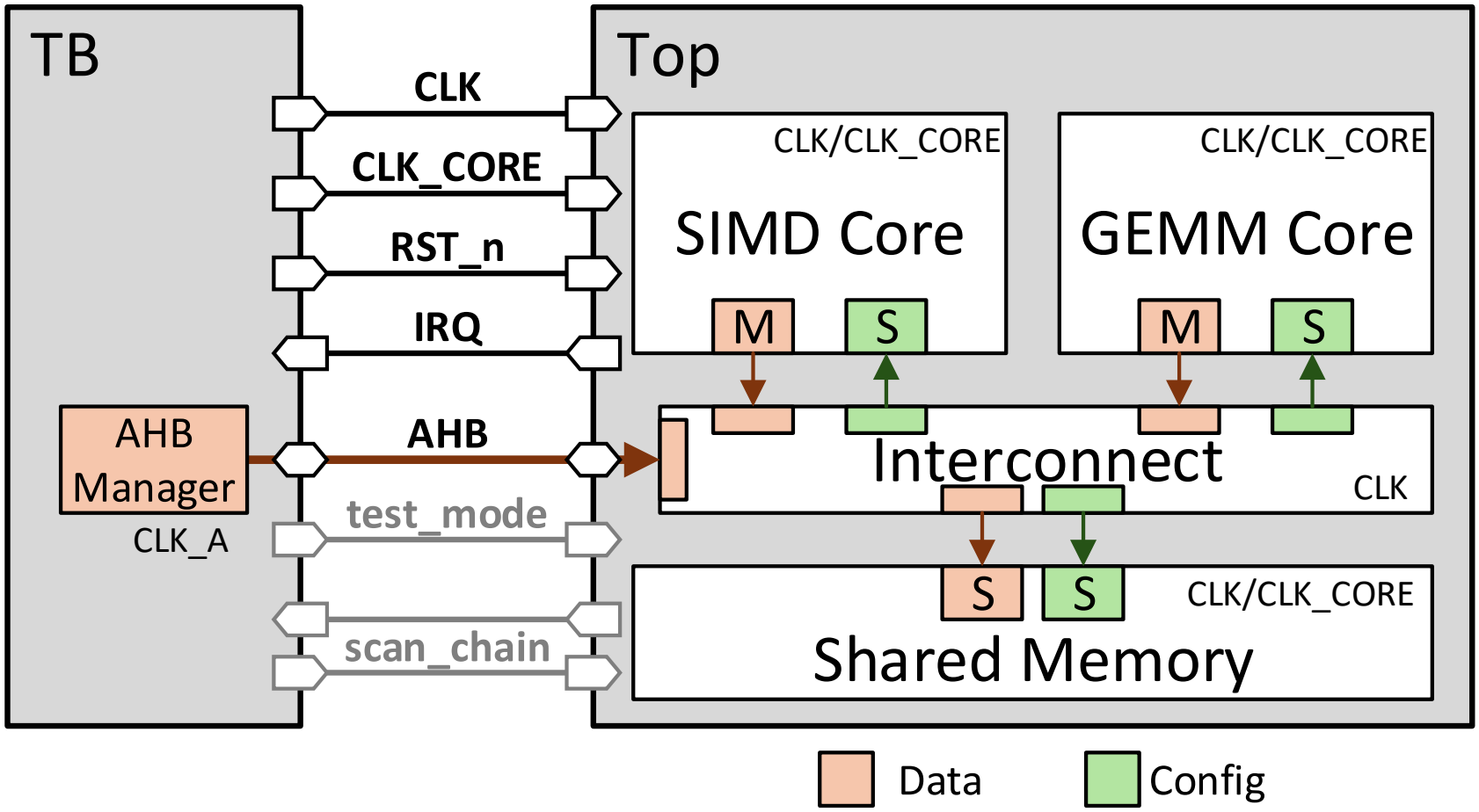
- In this homework, you will learn
  - How to insert scan chain
  - How to design and implement a small scale MBIST
  - How to combine the learned techniques
    - Clock Gating, DesignWare, Power Gating, CDC, Bus, DFT, etc.

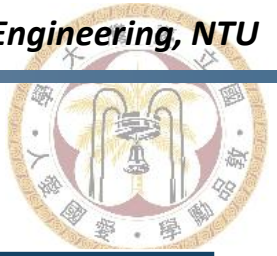


# Introduction

- Matrix operations are widely used in modern systems, including data analysis, deep learning and graphic computations.
- Single Instruction, Multiple Data (SIMD) is a type of parallel computing suitable for element-wise matrix operations.
- General Matrix Multiplication (GEMM) is a core computational kernel for matrix operations.
- You are asked to implement a system with a SIMD Core, a GEMM Core, and a Shared Memory connected through AHB.

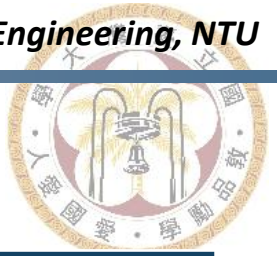
# Block Diagram





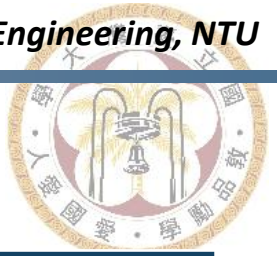
# Input/Output

| Signal Name | I/O | Width | Simple Description |
|---|---|---|---|
| **CLK** | I | 1 | Clock for IO and AHB, 100 MHz<br>All IOs are synchronized with the positive edge<br>Input delay: **1.0 ns**; Output delay: **0.5 ns** |
| **CLK_CORE** | I | 1 | Second clock for computational core or memory<br>Arbitrary frequency |
| **RST_N** | I | 1 | Active **low** asynchronous reset |
| **IRQ** | O | 1 | Interrupt signal, may initiate polling from TB |



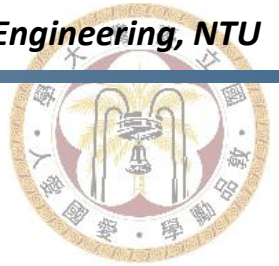
# Input/Output

| Signal Name | I/O | Width | Simple Description |
|---|---|---|---|
| HADDR | I | 14 | AHB address, 0x0000 – 0x3FFF |
| HWRITE | I | 1 | AHB write (0: read, 1: write) |
| HSIZE | I | 3 | AHB size, $2^{HSIZE}$ Byte per transfer |
| HBURST | I | 3 | AHB burst, only SINGLE and INCR are required |
| HTRANS | I | 2 | AHB transfer type |
| HWDATA | I | 1024 | AHB write data |



# Input/Output

| Signal Name | I/O | Width | Simple Description |
|---|---|---|---|
| HREADY | O | 1 | AHB ready |
| HRESP | O | 1 | AHB response |
| HRDATA | O | 1024 | AHB read data |
| test_mode | I | 1 | Active **high** test mode<br>Close all power switches, bypass all isolation cells |
| scan_enable | I | 1 | Active **high** scan enable |
| test_si/so | I/O | n | Scan chain input / output |

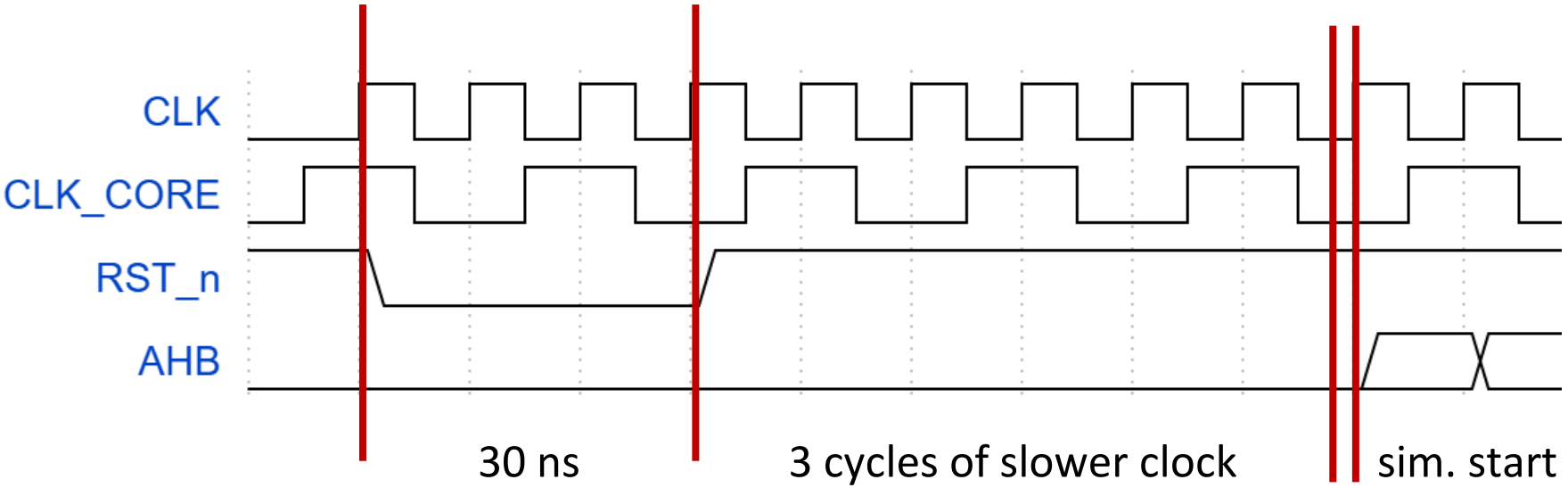


# Specification

- Active **low** asynchronous reset is used only once
- You should implement reset synchronizer for both clock
- You must implement the AHB protocol
- There must be **at least one power domain with power switch**
- There must be **at least one clock domain crossing**
- There must be **at least one scan chain**
- Most of the registers should be clock gated
- There must not be any latch (except clock gating cells)
- There must not be setup time violation

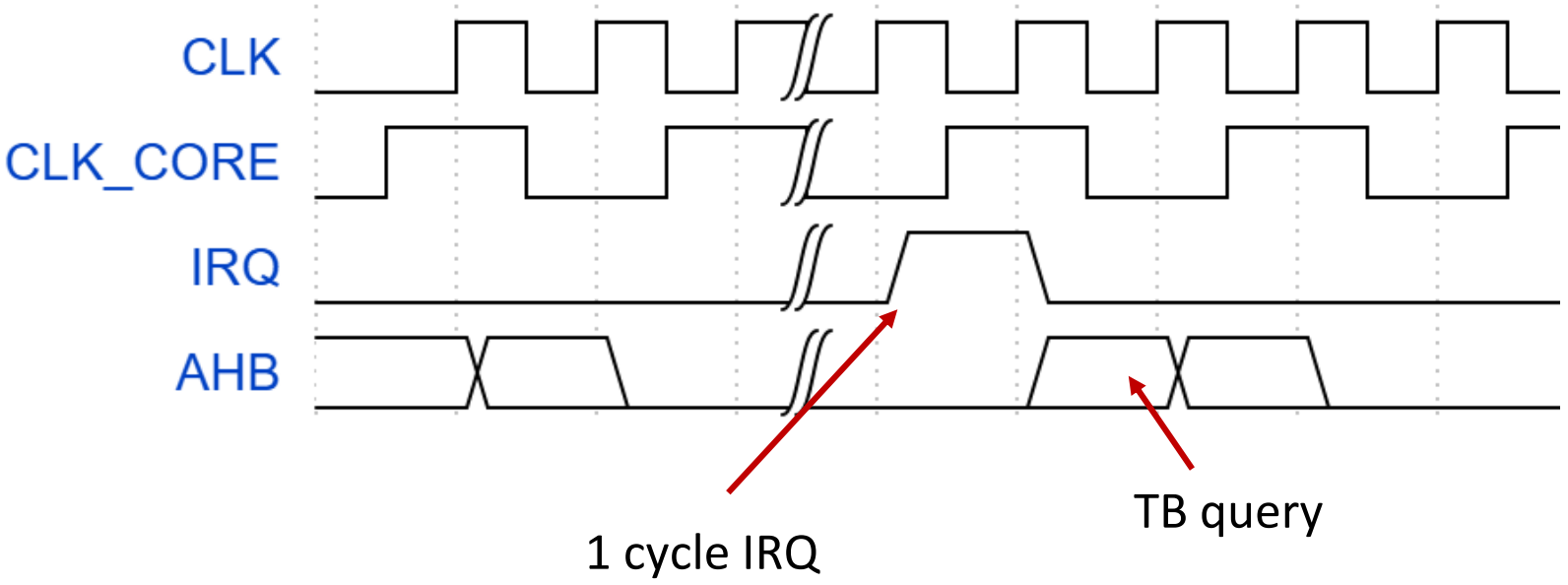
# Timing Specification (1/3)

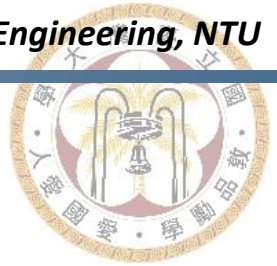
- RST_n will be low for 30 ns and be pulled high
- Simulation time starts from the next posedge of CLK after 3 cycles of the slower clock
- TB then tries to query the status and determine next step



# Timing Specification (2/3)

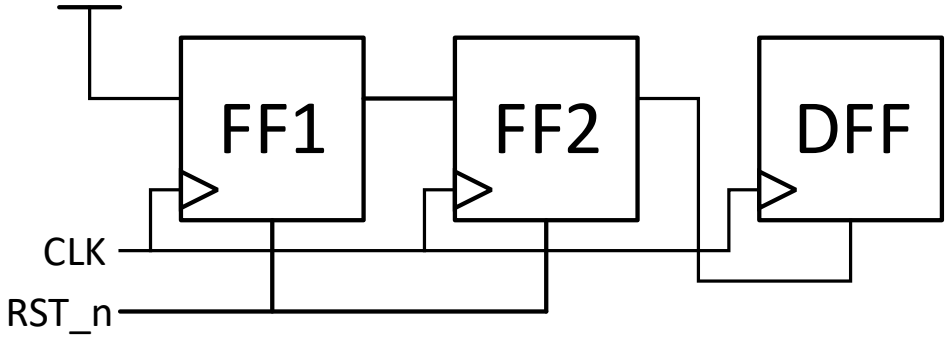
- TB will try to complete data transfer as fast as possible
- TB will randomly query status during Core execution
- A cycle of IRQ while waiting informs TB to check status
- Simulation time ends when all data is read out

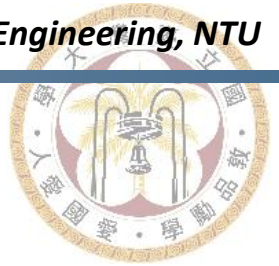




# Reset Synchronizer

- Asynchronous reset, synchronous release
- Fix reset timing violation due to multiple clock
- Similar to two stage synchronizer
- All DFF are written in asynchronous reset style

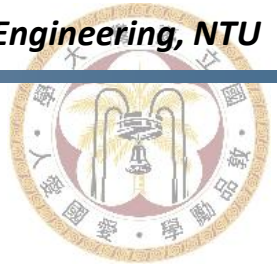




# Accelerator Status

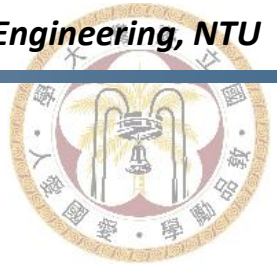
- Accelerator can be in one of the following status

| Status | Description |
|---|---|
| IDLE | No operation, TB might write/read config |
| DMA | Only TB access Shared Memory |
| SIMD | SIMD busy & access Shared Memory, TB might read config |
| GEMM | GEMM busy & access Shared Memory, TB might read config |
| MBIST | Shared Memory busy, TB might read config |
| SCAN | **`test_mode`** = 1, not tested in TB |



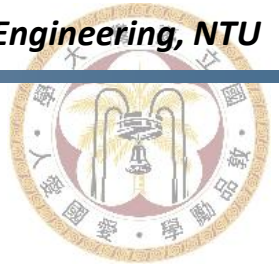
# TB Test Sequence

- There are 4 test sequences
  - DMA input > SIMD > DMA output
    - +define+SIMD
  - DMA input > GEMM > DMA output
    - +define+GEMM
  - MBIST > DMA input > SIMD > DMA output
    - +define+MBIST+SIMD
  - MBIST > DMA input > GEMM > DMA output
    - +define+MBIST+GEMM



# AHB Requirement

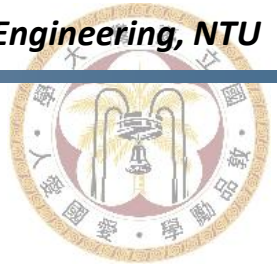
- Bus width = 1024 bits
- Support all possible **HSIZE**
- Support **SINGLE** and **INCR** burst mode
- Support all transfer type
- Support error response
- Burst from TB will be aligned and won't cross 1KB boundary
- A default subordinate should be implemented to handle request to unused address sections
  - Unused address: 0x1800 – 0x1FFF and 0x2C00 – 0x3FFF
  - SEQ or NONSEQ → ERROR response
    IDLE or BUSY → OKAY response



# AHB Data Layer

- Support up to 1024-bit, **SINGLE** and **INCR** mode
- TB only access memory data when all modules are deactivated
- SIMD and GEMM won't access memory data simultaneously
- Shared Memory should only need to process request from one manager at a time

| Module Name | Interface | Address |
|---|---|---|
| TB | M1 | - |
| SIMD | M2 | - |
| GEMM | M3 | - |
| Shared Mem | S4 | 0x0000 – 0x17FF |



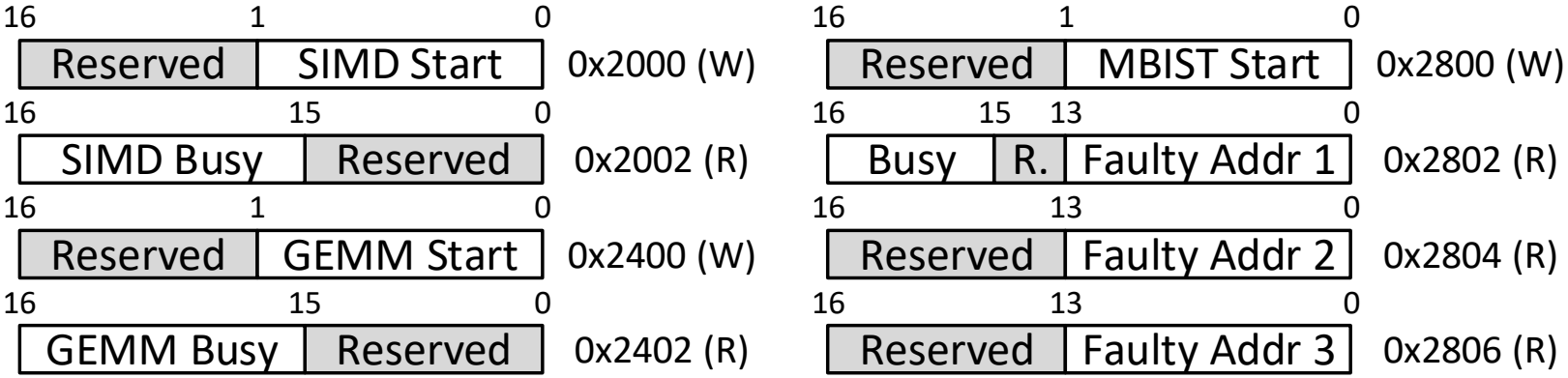
# AHB Config Layer (1/2)

- Support fixed 16-bit, **SINGLE** mode
- TB uses the same interface for data and config
- Shared Memory has separated data and config interface
- TB control SoC by accessing others' config register

| Module Name | Interface | Address |
|---|---|---|
| TB | M1 | - |
| SIMD | S1 | 0x2000 – 0x23FF |
| GEMM | S2 | 0x2400 – 0x27FF |
| Shared Mem | S3 | 0x2800 – 0x2BFF |

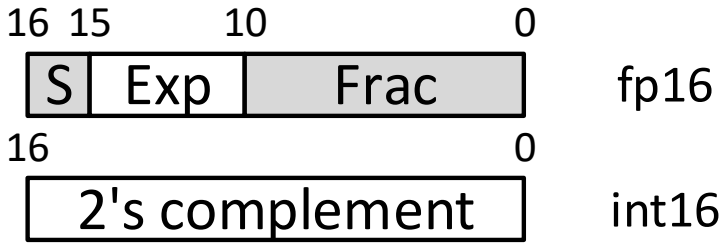
# AHB Config Layer (2/2)

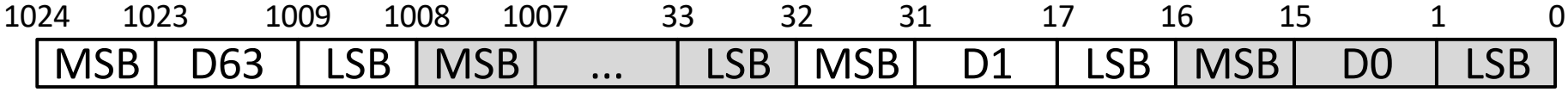
- TB will only write config when all modules are deactivated
- TB may read status from any module at any time
- TB only access the following config addresses
- **Reserved** section should hold 0's



# Data Format & Memory

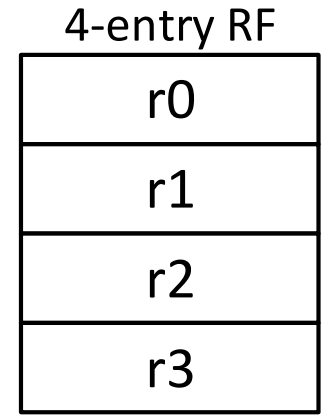
- Data width: 16-bit
- Little-endian
- Bus = 64 data = 1024 bit
- Total 6 KB = 3 x 32x32 x 16 bit
- The same bit pattern is interpreted as fp16 or 2's complement signed int16
- Fp16 operations follow IEEE 754 Std. and round to nearest even

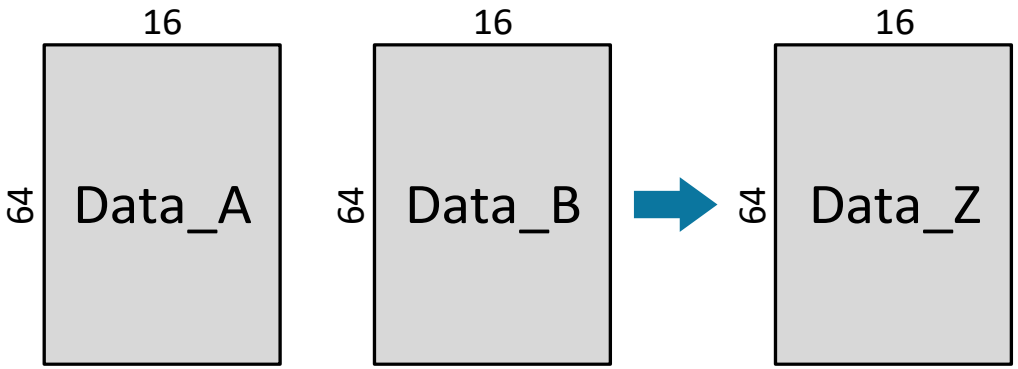


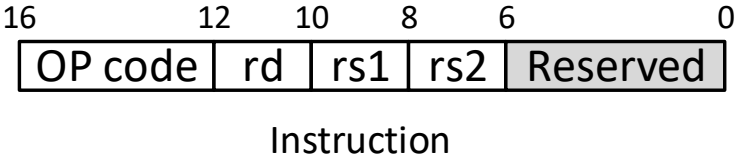



# SIMD Core

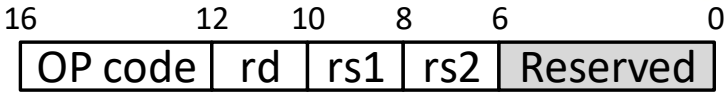
- Contains 16 parallel units, each with an ALU and an 4-entry RF
- Computes element-wise operations
- Execute 16 instructions on each data pair
- Each instruction has 16 bits



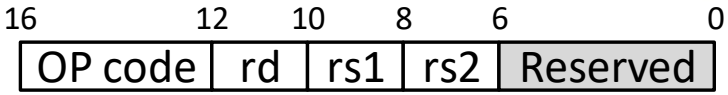
# SIMD Instructions (1/2)




- 16 instructions

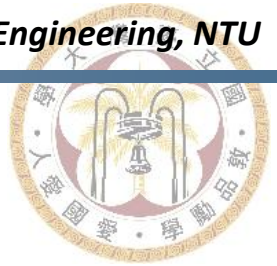
| Name | OP Code | Operation |
|---|---|---|
| ADD | 0000 | rd = rs1 + rs2 |
| SUB | 0001 | rd = rs1 – rs2 |
| MUL | 0010 | rd = rs1 × rs2 |
| Less Than | 0011 | rd = (rs1 < rs2)<br>True: 0101…01, False: 1010…10 |
| Shift Left | 0100 | rd = rs1 << rs2[3:0] |
| Arith. Shift Right | 0101 | rd = rs1 >>> rs2[3:0] |
| Bitwise NOT | 0110 | rd = ~rs1 |
| Bitwise OR | 0111 | rd = rs1 \| rs2 |

# SIMD Instructions (1/2)




- 16 instructions

| Name | OP Code | Operation |
|---|---|---|
| Bitwise AND | 1000 | rd = rs1 & rs2 |
| Bitwise XOR | 1001 | rd = rs1 ^ rs2 |
| FP ADD | 1010 | rd = rs1 + rs2 |
| FP SUB | 1011 | rd = rs1 – rs2 |
| FP MUL | 1100 | rd = rs1 × rs2 |
| FP DIV | 1101 | rd = rs1 / rs2 |
| FP Less Than | 1110 | rd = (rs1 < rs2)<br>True: 0101…01, False: 1010…10 |
| STORE | 1111 | data_z = rd |



# SIMD Operation

- Start execution when SIMD Start is asserted
- Unset SIMD Start immediately and set SIMD Busy until finish
- Optionally pull IRQ high for 1 cyc to inform TB when finish

1. Load 16 new instructions
2. Clear all register file entry (set to 0's)
3. Load $Data_a_i$ to [r0] and $Data_b_i$ to [r1]
4. Execute 16 instructions
5. Store $Data_z_i$

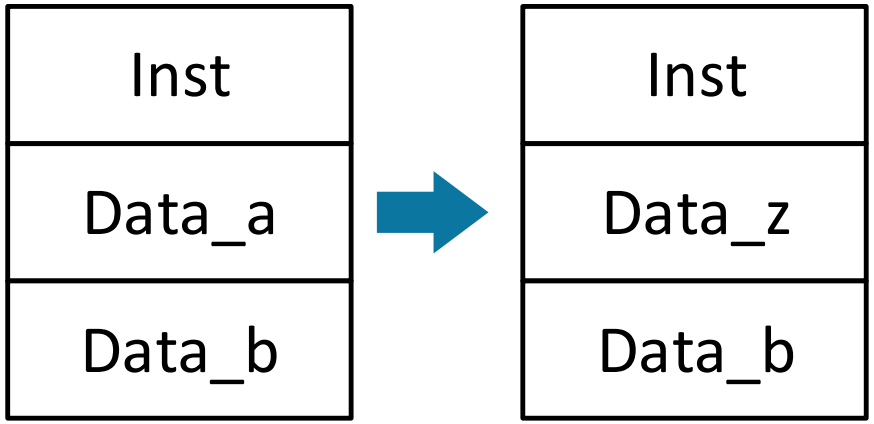
- There is only one STORE instruction every round, and it may not necessarily be the last one

# SIMD Memory Mapping

- Data is stored in row-major order
- Instruction: 0x0000 – 0x07FF
- $Data_a$: 0x0800 – 0x0FFF
- $Data_b$: 0x1000 – 0x17FF

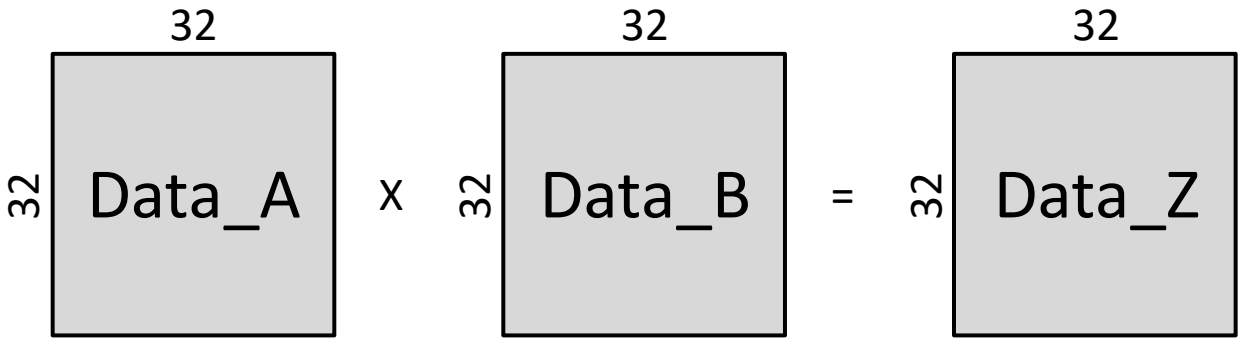
| | | | | |
|---|---|---|---|---|
| 0 | 1 | 2 | ... | 15 |
| 16 | 17 | 18 | ... | 31 |
| 32 | 33 | 34 | ... | |

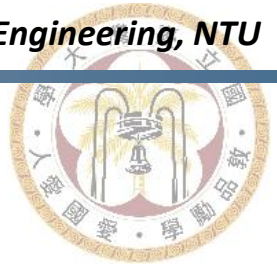
- $Data_z$: 0x0800 – 0x0FFF (override $Data_a$)



# GEMM Core

- Contains 32 MAC units
- Computes general matrix multiplication
- All data and intermediate values are fp16
- It is recommended to cache few rows to reduce bandwidth





# GEMM Operation

- Start execution when GEMM Start is asserted
- Unset GEMM Start immediately and set GEMM Busy until finish
- Optionally pull IRQ high for 1 cyc to inform TB when finish

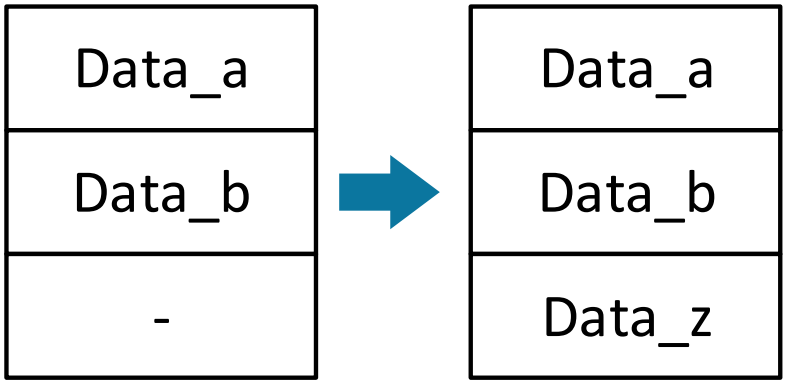
1. Cache N rows of $data_a$
2. Load 1 col of $data_b$
3. Execute N dot product
4. Repeat from 2
5. Store N rows of $data_z$

# GEMM Memory Mapping

- $Data_a$ and $Data_z$ is stored in row-major order
- $Data_b$ is stored in col-major order
- $Data_a$: 0x0000 – 0x07FF
- $Data_b$: 0x0800 – 0x0FFF
- $Data_z$: 0x1000 – 0x17FF

$Data_a/z$

| | | | | |
|---|---|---|---|---|
| 0 | 1 | 2 | … | 31 |
| 32 | 33 | 34 | … | 63 |
| 64 | 65 | 66 | … | |

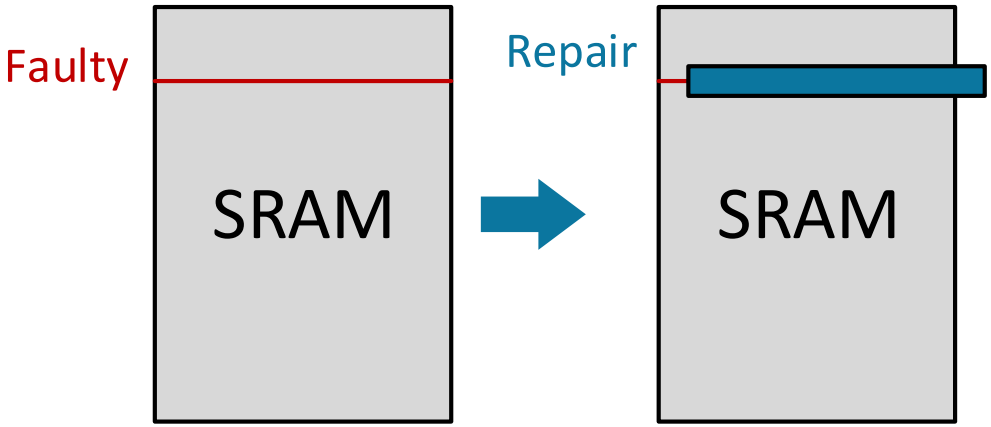


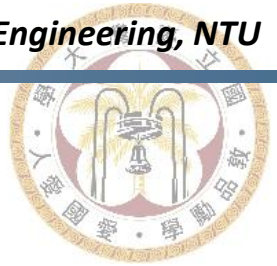

$Data_b$

| | | |
|---|---|---|
| 0 | 32 | 64 |
| 1 | 33 | 65 |
| 2 | 34 | 66 |
| ⋮ | ⋮ | ⋮ |
| 31 | 63 | |

# Shared Memory

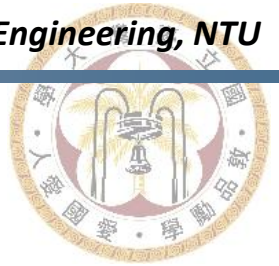
- Capacity: 6 KB or above
- TB will only use the first 6 KB (0x0000 – 0x17FF)
- Must implement your own simple MBIST and redundancy
- At most 3 stuck-at faults spread across 6 KB
- Replace faulty site with redundancy storage such that the repairment is transparent for user





# Shared Memory Operation

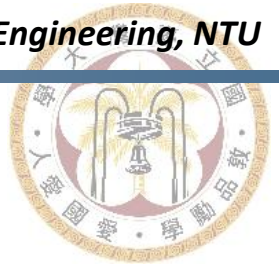
- Start MBIST when MBIST Start is asserted
- Unset MBIST Start immediately and set MBIST Busy until finish
- Present the faulty address starting from Faulty Addr1
- Optionally pull IRQ high for 1 cyc to inform TB when finish
- TB will check the result any time after MBIST Busy is unset
- If less than 3 faulty address, fill the Faulty Addrx with 1's
- If multiple faults are within one address, only present the faulty address once
- The order of the Faulty Addrx can be arbitrary



# Memory Module

- All memory in Shared Memory must be instantiate from SRAM/spsramXXX or SRAM/dprfXXX provided by TA

```
01_RTL/SRAM:
dprf128x32m4.v  dprf16x120m2.v  dprf16x72m2.v  spsram128x64m4.v  spsram16x96m2.v
dprf128x64m4.v  dprf16x32m2.v   dprf32x32m2.v  spsram16x88m2.v   spsram512x45m4.v
```



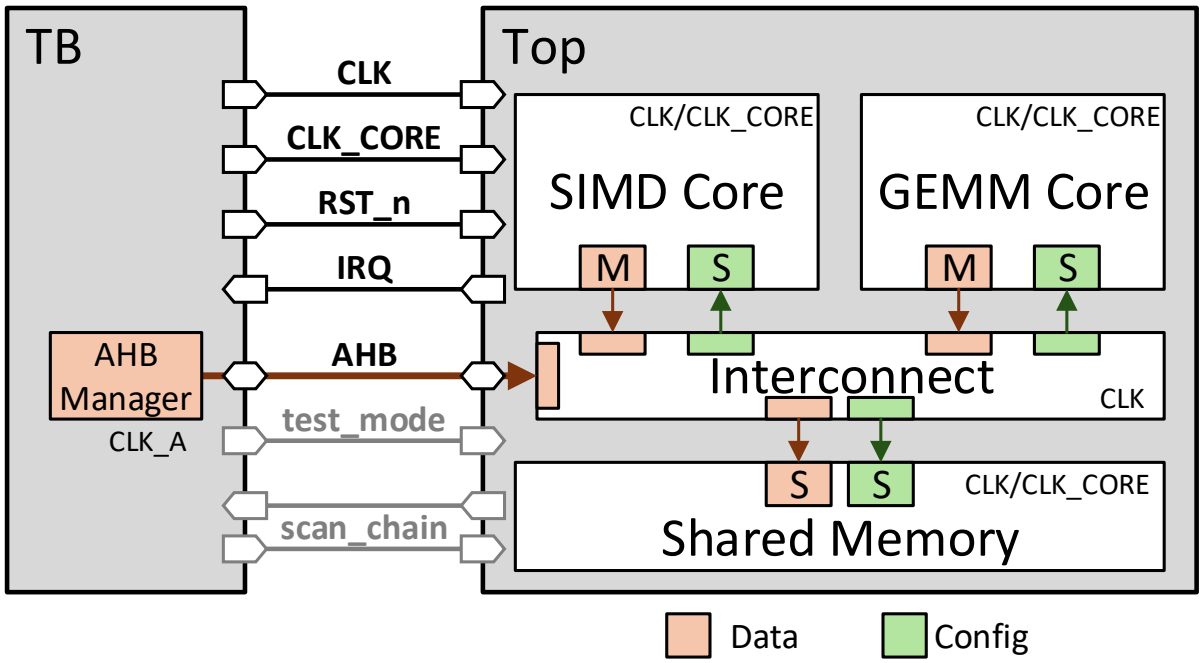
# Scan Chain & ATPG

- You must insert **at least one scan chain** in your design
- All registers must be on the scan chain(s)
- **Use dedicate scan in & out pin**

- You must also generate test pattern with tmax

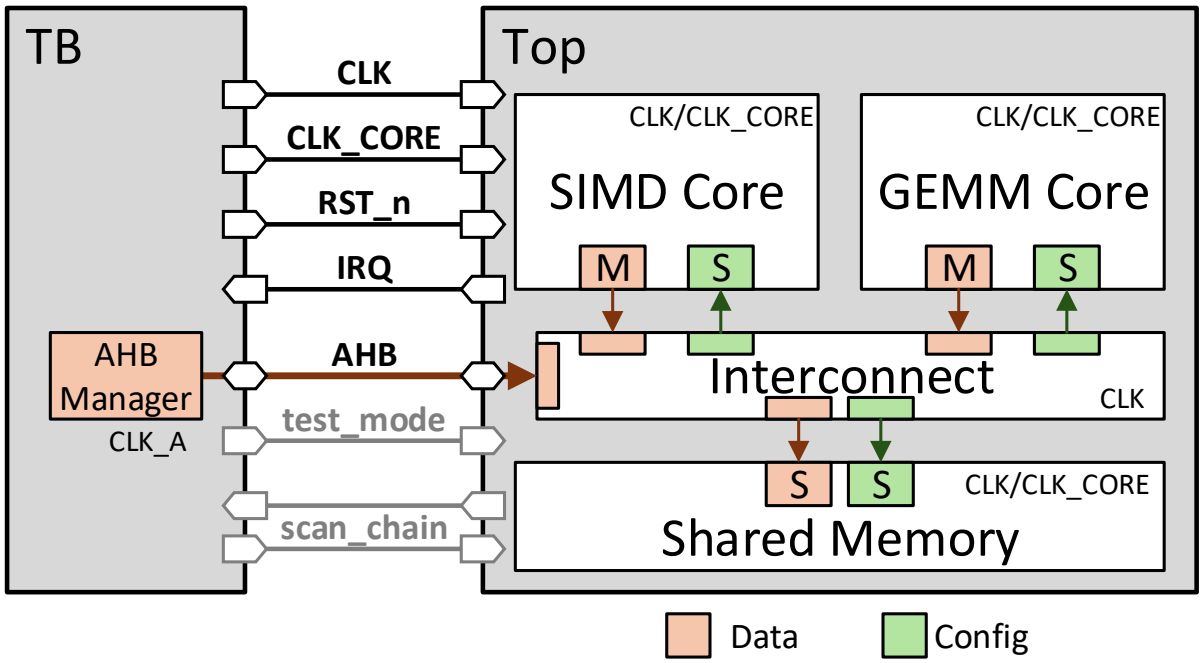
# Power Gating

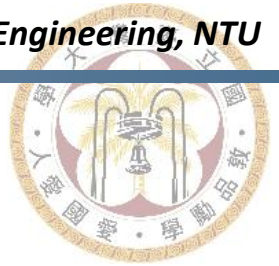
- Operation of SIMD and GEMM is exclusive, it would be a waste of power if we don't turn off the other
- You must implement power gating
- Setting power gating for both computation core is a good idea



# Clock Domain Crossing

- The SoC interface and AHB should be operating at 100 MHz, which is very slow for computation core
- You must implement multi-clock design with CDC
- Placing CDC right before computation core is a good idea

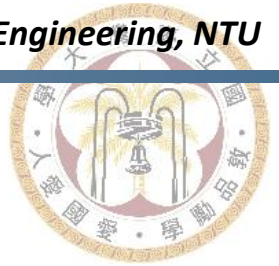




# Other Requirement (1/2)

- You are asked to compare the effect of different techniques
- Synthesize the following 5 versions with different features

| Feature | Baseline | DFT | PD | Multi-Clock | ALL |
|---|---|---|---|---|---|
| SIMD & GEMM | O | O | O | O | O |
| MBIST & Scan Chain | | O | O | O | O |
| Power Gating | | | O | | O |
| CDC | | | | O | O |



# Other Requirement (2/2)

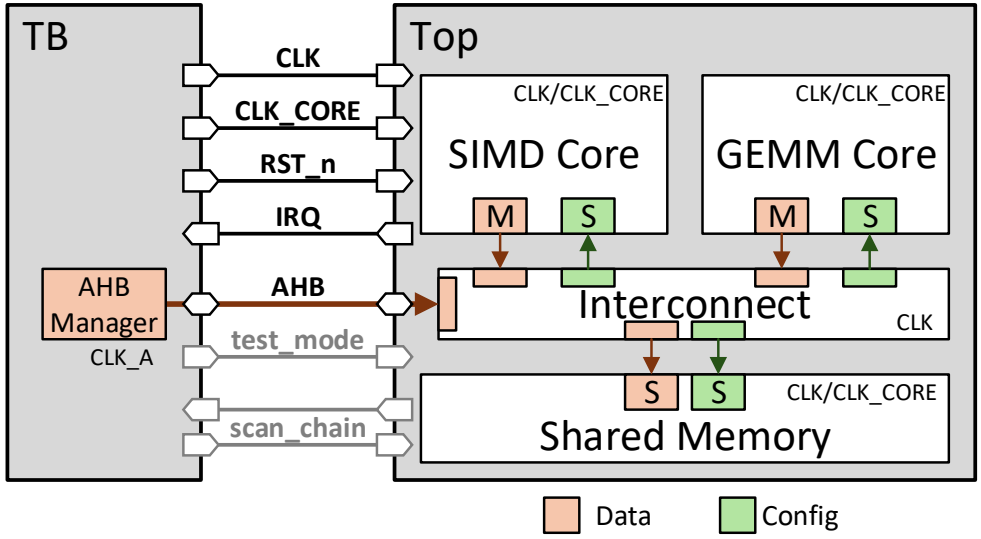
- Q1. List their hierarchical area, and simulation time and power of each test sequences in a table
- Q2. Compare the above metrics between the following versions
  1. Baseline vs DFT
  2. DFT vs PD
  3. DFT vs Multi-Clock
- Q3. Select the best version to be used in performance ranking
- Q4. The screenshot of the VCS simulation result of ATPG patterns

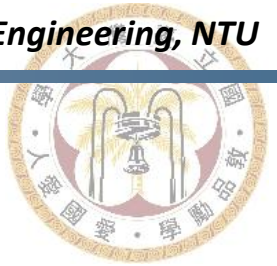
- See report.docx for details

# Preserve Hierarchy

- Include the following command in the synthesis script to presser hierarchy of major modules

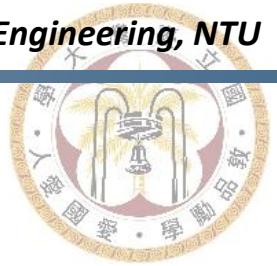
```
set_ungroup [get_designs SIMD_Core*    ] false
set_ungroup [get_designs GEMM_Core*    ] false
set_ungroup [get_designs Shared_Memory*] false
set_ungroup [get_designs Interconnect* ] false
```





# Hint

- The scripts may not be complete. Remember to modify them
- You CAN use DesignWare in this homework
- Do not spend too much time on circuit optimization
- If your design cannot meet hold time requirements even after `compile -inc -only_hold_time`, you can run gate sim with +notimingcheck
  - **State it clearly in your report!!**



# top.sv

```
1   module top # (
2       parameter   DATA_WIDTH = 16
3   ) (
4       input   logic   CLK,
5       input   logic   CLK_CORE,
6       input   logic   RST_N,
7       output  logic   IRQ,
8
9       input   logic   [  13:0] HADDR,
10      input   logic            HWRITE,
11      input   logic   [   2:0] HSIZE,
12      input   logic   [   2:0] HBURST,
13      input   logic   [   1:0] HTRANS,
14      input   logic   [1023:0] HWDATA,
15      output  logic            HREADY,
16      output  logic            HRESP,
17      output  logic   [1023:0] HRDATA,
18
19      input   logic   test_mode,      // for PMU bypass
20      input   logic   scan_enable     // for DFT, not used in RTL
21  );
```

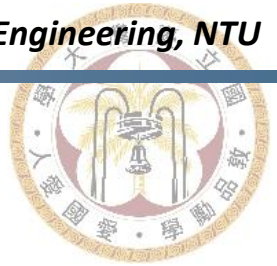
# 01_run & rtl.f

VCS command:

vcs -full64 -R -file rtl.f +v2k -sverilog -debug_access+all +notimingcheck +define+RTL+FSDB+$1 | tee sim_$1.log

```
1   // Testbench
2   ../00_TB/tb.sv
3
4   // DesignWare
5   // -y /path/to/dw/
6   // +libext+<file extension>
7   // +incdir++/path/to/dw/
8
9   // UPF
10  // -power
11  // -upf ../01_RTL/top.upf
12
```

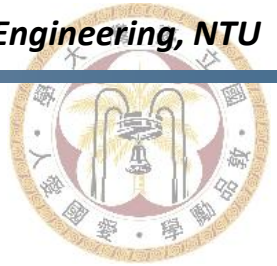
```
13  // SRAM
14  // -v /path/to/sram/SRAM.v
15  -y ../01_RTL/SRAM/
16
17  // Design files
18  ../01_RTL/top.sv
19  ../01_RTL/SIMD_Core.sv
20  ../01_RTL/GEMM_Core.sv
21  ../01_RTL/Shared_Memory.sv
22  ../01_RTL/Interconnect.sv
23
```



# report_all.txt

- Clock period for CLK_CORE
- Total number of memory instances in Shared Memory
- Total cell area after scan chain insertion
- Gate-level runtime and power for MBIST+SIMD and MBIST+GEMM

```
1   Student ID: r13943xxx
2
3   Clock period of CLK_CORE  : x.x ns
4   Number of memory instance : xxx
5
6   ------------------------------
7   Area: xxx.xxxxxx um^2
8   ------------------------------
9   MBIST+SIMD time : xxx.xxx ns
10  MBIST+SIMD power: xxx.xxx mW
11  ------------------------------
12  MBIST+GEMM time : xxx.xxx ns
13  MBIST+GEMM power: xxx.xxx mW
```



# report_ranking.txt

- Clock period for CLK_CORE
- Total number of memory instances in Shared Memory
- Total cell area after scan chain insertion
- Gate-level runtime and power for MBIST+SIMD and MBIST+GEMM

```
1   Student ID: r13943xxx
2
3   Clock period of CLK_CORE  : x.x ns
4   Number of memory instance : xxx
5
6   ------------------------------
7   Version for performance ranking: DFT | PD | Multi-Clock | ALL
8   Area: xxx.xxxxxx um^2
9   ------------------------------
10  MBIST+SIMD time : xxx.xxx ns
11  MBIST+SIMD power: xxx.xxx mW
12  ------------------------------
13  MBIST+GEMM time : xxx.xxx ns
14  MBIST+GEMM power: xxx.xxx mW
```

# report.pdf

ACVSD HW3 Report

ID: r13943xxx

ADFP account: acvsd-xx

1. For each version, list its hierarchical area, and simulation time and power of each test sequences in the table below.
   Task: A-SIMD, B-GEMM, C-MBIST+SIMD, D-MBIST+GEMM
   Units: um², ns, uW/mW (select what you use, round to integer)

| | Baseline | DFT | PD | Multi-Clock | ALL |
|---|---|---|---|---|---|
| Area-total | | | | | |
| Area-SIMD | | | | | |
| Area-GEMM | | | | | |
| Time-A | | | | | |
| Time-B | | | | | |
| Time-C | | | | | |
| Time-D | | | | | |
| Power-A | | | | | |
| Power-B | | | | | |
| Power-C | | | | | |
| Power-D | | | | | |

2. Compare the following pair, what are the effect of each technique?
   Briefly explain in 1-3 sentences.
   - Baseline vs DFT
   - DFT vs PD
   - DFT vs Multi-Clock

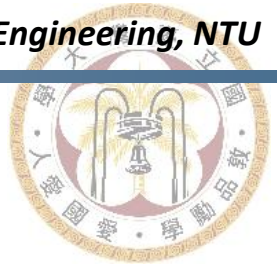
3. Which version did you choose for performance ranking? Why does it perform better than other versions? Briefly explain in 1-3 sentences.

4. Attach the screenshot of the VCS simulation result for ATPG patterns. The screenshot should include the line "Simulation of xxx patterns completed with 0 mismatches ..." and the next command prompt, which shows your ADFP account.
   Note: TA has refined the bash script for you. You can now run the script directly using `./gen_atpg_tb.sh`, and `./top_atpg_vcs.sh` without changing to bash shell.

# Submission

- Create a folder named studentID_hw3 and follow the hierarchy below

```
r13943xxx_hw3
    ├── ALL
    │   ├── 01_RTL
    │   │   ├── 01_run
    │   │   ├── rtl.f
    │   │   ├── top.sv
    │   │   ├── xxx.sv (other RTL files)
    │   │   └── top.upf
    │   ├── 02_SYN
    │   │   ├── syn.tcl
    │   │   ├── top_syn.area (hierarchy)
    │   │   ├── top_syn.timing
    │   │   └── top_syn.scan_path
    │   ├── 03_ATPG
    │   │   ├── top_syn.scandef
    │   │   ├── top_syn.spf
    │   │   └── top_atpg.rpt
    │   ├── 04_GATE
    │   │   ├── 04_run
    │   │   ├── gate.f
    │   │   ├── top_syn.v
    │   │   ├── top_syn.sdf
    │   │   └── top_syn.upf
    │   ├── 05_PWR
    │   │   ├── top_SIMD.power
    │   │   └── top_GEMM.power
    │   └── report_all.txt
    ├── Ranking
    │   ├── 01_RTL - 05_PWR
    │   │   └── (same files except unnecessary ones)
    │   └── report_ranking.txt
    └── report.pdf
```



# Submission - Workstation

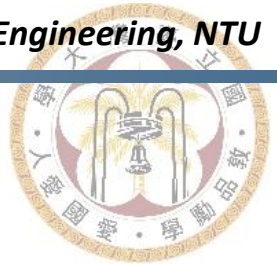
- Pack the folder **studentID_hw3** into a **tar.gz** file named **acvsd-xx-hw3.tar.gz**
  - **tar -zcvf acvsd-xx-hw3.tar.gz [path to studentID_hw3]**
  - Use **lowercase** for all the letters (e.g. acvsd-00-hw3.tar.gz)
  - Pack the folder on ADFP server to avoid OS related problems
  - Place the tar.gz file at the home dir of your ADFP account

- TA will only check the last version after the homework deadline
- Remember to keep a copy of the files uploaded to ADFP server on your local computer

# Submission – NTU COOL

- For design files (01_RTL and reports)
- Pack the folder **studentID_hw3** into a **tar.gz** file named **acvsd-xx-hw3-v*k*.tar.gz** (*k* is is the number of version, *k* =1,2,...)
  - **tar -zcvf acvsd-xx-hw3-v*k*.tar.gz [path to studentID_hw3]**
  - Use **lowercase** for all the letters (e.g. acvsd-00-hw3-v1.tar.gz)
- Submit to NTU COOL

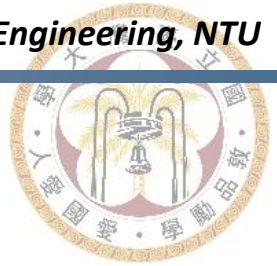
```
r13943xxx_hw3
    ├── ALL
    │   ├── 01_RTL
    │   │   ├── 01_run
    │   │   ├── rtl.f
    │   │   ├── top.sv
    │   │   ├── xxx.sv (other RTL files)
    │   │   └── top.upf
    │   └── report_all.txt
    ├── Ranking
    │   ├── 01_RTL
    │   │   └── (same files except unnecessary ones)
    │   └── report_ranking.txt
    └── report.pdf
```



# Grading Policy

- TA will use **01_run** and **04_run** to run your code at RTL and gate-level simulation. Modify them to your needs.
  - e.g. 01_run SIMD, 04_run MBIST+GEMM
- Simulation (60%)
  - 4 sequences, one quarter each
  - Pass all 4 sequences to gain any point from next level

| Score | Baseline | DFT | PD | Multi-Clock | ALL |
|---|---|---|---|---|---|
| RTL public | 5% | 10% | 15% | 15% | 20% |
| Gate-level public | 5% | 10% | 15% | 15% | 20% |
| Gate-level private | 5% | 10% | 15% | 15% | 20% |



# Grading Policy

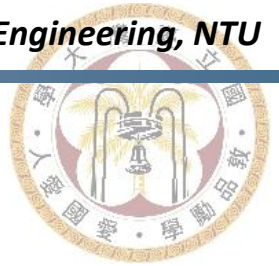
- Word report (10%)
- Performance ranking (30%)
  - **Your score will be halved if you didn't pass all simulations**
  - **You are excluded if you didn't pass Gate-level public**

$$\text{Score} = A \times \left( \begin{matrix} T_{MBIST+SIMD} \times P_{MBIST+SIMD} \\ +T_{MBIST+GEMM} \times P_{MBIST+GEMM} \end{matrix} \right)$$

**A: Total cell area ($um^2$)**

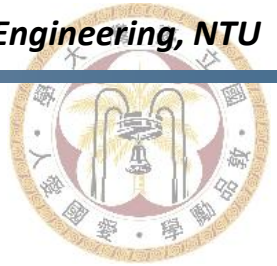
**$T_x$: Simulation time of mode *x* reported by testbench (ns)**

**$P_x$: Average power of mode *x* reported by PrimeTime (mW)**



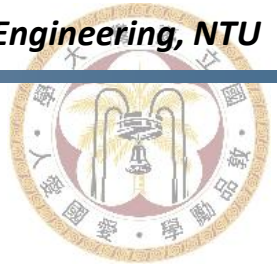
# Grading Policy

- Do not memorize the answers directly in any way
- Lose **5 points** for any wrong naming rule or format for submission
- **No delay submission is allowed**
- **No plagiarism**



# Discussion

- NTU Cool Discussion Forum
  - For any questions not related to assignment answers or privacy concerns, please use the NTU Cool discussion forum.
  - **TAs will prioritize answering questions on the NTU Cool discussion forum**
- Email: r13943005@ntu.edu.tw
  - Title should start with [ACVSD 2026 Spring HW3]
  - Email with wrong title will be moved to trash automatically



# References

[1] AMBA AHB Protocol Specification
https://www.arm.com/zh-tw/architecture/system-architectures/amba/amba-specifications

[2] Synopsys ® Multivoltage Flow User Guide

[3] R. Ginosar, Metastability and Synchronizers: A Tutorial, 2011.

[4] Clifford E. Cummings, Clock Domain Crossing (CDC) Design & Verification Techniques Using SystemVerilog, 2008.